ZEWAIL CITY OF SCIENCE AND TECHNOLOGY School of Computational Sciences and Artificial Intelligence (CSAI)

ACADEMIC COURSE & SCHEDULE PLANNERVERSION

PHASE 2

ABSTRACT

This SRS document outlines the requirements for the Academic Course & Schedule Planner, a web platform for automated timetable generation, prerequisite verification, and academic advising. Built using an MVC architecture, it provides role-based portals, conflict detection, and GPA tracking to streamline university course registration.

Prepared By:

Student Name	Student ID
Badr Mostafa Ahmed	202507541
Yousef Mahmoud Essmat	202506278
Ziad Mohamed Ahmed	202508680

Submitted To: Dr. Mohamed Sami Rakha & Course Teaching Assistants: Mohamed Nabil

Date of Submission: 30/10

Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document details the complete functional and non-functional requirements for the Academic Course & Schedule Planner, Version 1.0. The primary purpose of this document is to serve as a clear, definitive contract between the software development team, academic advisors, and system administrators. It establishes a shared baseline for the subsequent software engineering phases, specifically system architecture design, database schema implementation, unit testing, and final project evaluation.

1.2 Scope

The Academic Course & Schedule Planner is a centralized web application designed to simplify semester planning for university students, academic advisors, and department administrators.

- Key Capabilities: The software provides automated, conflict-free timetable generation, real-time prerequisite/co-requisite verification, GPA calculation and target prediction, degree progress auditing, and an online schedule submission system for academic advisor review. Additionally, administrators are provided with tools to manage course catalog offerings, seat capacities, instructor assignments, and bulk data imports.
- Out of Scope: The system will not handle direct tuition fee payment processing, physical classroom key allocations, or campus library book borrowing, as these fall under separate university enterprise systems.

1.3 Definitions, Acronyms, and Abbreviations

- SRS: Software Requirements Specification.
- SDLC: Software Development Life Cycle.
- MVC: Model-View-Controller Architectural Pattern.
- GPA / CGPA: Grade Point Average / Cumulative Grade Point Average.
- 2FA: Two-Factor Authentication.
- UI/UX: User Interface / User Experience.
- API: Application Programming Interface.
- JSON / CSV: JavaScript Object Notation / Comma-Separated Values.
- FR: Functional Requirement
- NFR: Non-Functional Requirement
- URL: Uniform Resource Locator
- SUS: System Usability Scale

1.4 References

1. IEEE Computer Society, IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications.
2. Ian Sommerville, Software Engineering, 10th Edition, Pearson, 2015.
3. Zewail City of Science and Technology, CSAI 203 Course Project Guidelines and Requirements, Fall 2026.
4. Roger S. Pressman, Software Engineering: A Practitioner's Approach, McGraw-Hill.

1.5 Overview

This SRS document is structured into four primary sections:

- Introduction: Provides the document purpose, scope, definitions, references, and overall organization.
- Overall Description: Presents a high-level overview of product perspective, user classes, operational environment, and design constraints.
- Specific Requirements: Details the 20 functional requirement features, UML use case model, domain class diagram, 5 non-functional requirement test plans, and external interfaces.
- Appendices: Contains the system data dictionary and complete glossary.

Overall Description

2.1 Product Perspective

The Academic Course & Schedule Planner is a standalone, web-based software application. While operating independently, it is designed with flexible API boundaries to allow future integration with existing University Student Information Systems (SIS) and Authentication portals. The system follows a strict Model-View-Controller (MVC) software architecture to ensure clean separation between data logic, presentation layers, and user interactions.

2.2 Product Functions

The high-level functionalities provided by the system are grouped into five core operational modules:

1. User Authentication & Profile Management: Role-based access (Student, Advisor, Admin), profile customization, and degree plan setup.
2. Interactive Schedule Builder & Conflict Engine: Drag-and-drop timetable grid, automated time-clash detection, prerequisite checking, and local state persistence.
3. Academic Progress & Analytics: Cumulative/Semester GPA calculation, grade target estimation, and major degree audit tracking.

4. **Advisor Interaction Workflow: Schedule draft submission, advisor review queue, approval/rejection comments, and overload credit requests.**
5. **System Administration & Catalog Management: CRUD operations for course offerings, seat capacity adjustments, waitlist management, 2FA admin authentication, and CSV/JSON bulk data processing.**

2.3 User Classes and Characteristics

- **Student (Primary User): Undergraduate students who use the platform to search courses, plan conflict-free schedules, evaluate GPAs, and submit plans for approval. Requires minimal technical training.**
- **Academic Advisor (Secondary User): Faculty members who review student schedule submissions, inspect academic history, and approve or request schedule revisions.**
- **System Administrator (Tertiary User): Academic affairs staff responsible for maintaining course offerings, managing room and instructor allocations, monitoring section capacities, and importing semester schedules.**

2.4 Software Stakeholders

- **Students: Primary beneficiaries seeking an efficient, error-free registration and schedule planning experience.**
- **Academic Advisors & Faculty: Benefit from streamlined schedule review processes and reduced administrative overhead.**
- **University Registrar & Academic Affairs: Interested in accurate enrollment tracking, room capacity management, and waitlist prioritization.**
- **Development Team & Course Instructors: Interested in verifying sound software engineering principles, proper requirements traceability, and adherence to project guidelines.**

2.5 Operating Environment

- **Client Side: Modern web browsers (Google Chrome 90+, Mozilla Firefox 88+, Microsoft Edge, Safari) operating on desktops, tablets, or smartphones with a minimum horizontal resolution of 360px.**
- **Server Side: Web application server environment running Python (Flask framework) or C# (.NET Core).**
- **Database Layer: Relational Database Management System (RDBMS) such as PostgreSQL or SQLite.**

2.6 Design and Implementation Constraints

- **Architectural Mandate: Mandatory application of the Model-View-Controller (MVC) software pattern.**
- **Document Guidelines: The SRS document must strictly range between 12 to 17 pages, formatted in Times New Roman or Arial (12pt font, 1.5 line spacing).**
- **Version Control: All source files, documentation, and design assets must be maintained in a shared GitHub repository using proper branch workflows and Pull Requests.**
- **No Auto-Code Generation: Auto-code generation tools (UI-to-code or diagrams-to-code) are strictly prohibited.**

2.7 User Documentation

The application will provide built-in user guidance, including:

- **An interactive Frequently Asked Questions (FAQ) page covering schedule building, prerequisite resolution, and overload request submission.**
- **Contextual tooltips and inline error indicators explaining time conflicts or missing prerequisite requirements.**

2.8 Assumptions and Dependencies

- **Assumptions: Users have access to an active internet connection and standard web browsing devices. It is assumed that official course offerings and department prerequisite rules provided by admins are accurate.**
- **Dependencies: System schedule persistence depends on client-side browser LocalStorage availability for offline recovery.**

3.1 Functional Requirements

FR-01: User Login and Role Based Dashboards

Description: Users can log in using their email and password. There are three types of users within the system: Students, Academic Advisors and Administrators. Once a user logs in they will be automatically directed to their unique dashboards based on their permissions.

Scenario:

Main: A student successfully logs in using their email and password. Upon logging in their personal schedule page will load.

Alternative: An Academic Advisor logs in and will be automatically redirected to the student review page.

Exception: User has entered the incorrect password three times, their account will now be locked for 15 minutes and an error message will appear.

FR-02: Student Profile

Description: Students can set up their profile with details such as their major subject, minor subject, catalog year and credit hours. This information will be used to determine the students course pre-requisites and requirements.

Scenario:

Main: A new student completes their profile with major subject specified as Computer Science and Catalog Year specified as 2024-2025. The information will be stored, and their degree plan will be updated.

Alternative: A student updates their minor subject to Data Science from Math. The elective course list will be updated.

Exception: A student attempts to select a major that is no longer available for that catalog year. The system will stop them adding the course and ask them to select a valid major.

FR-03: Search Courses

Description: Users can search for available courses. The search page allows users to filter results by course code, course name, department, credit hours, instructor, and class time.

Scenario:

Main: A student filters their results by entering CSAI in the department field and selecting No Friday Classes. The search results will display relevant courses with open spaces and times.

Alternative: A student types in part of the course name e.g. Data Struct. The system will automatically find CSAI 201: Data Structures.

Exception: The search results do not return any courses. An error message will appear displaying No courses found. A clear button will also be provided to clear all filters.

FR-04: Check Pre-requisites and Co-requisites

Description: Before a student can add a course to their schedule, the system will automatically check if they have passed the required pre-requisite courses with the required grade or if they need to add a co-requisite lab.

Scenario:

Main: A student attempts to add course CSAI 203. The system will check their history and see that they have passed CSAI 201 with a grade of C or higher and will add the course to their draft.

Alternative: A student adds a course that requires a co-requisite lab. The system will add the lecture and prompt the student to add a lab section.

Exception: A student attempts to add a course but hasn't taken the pre-requisite course. The system will stop them and tell them what course they are missing.

FR-05: Create Timetable Manually

Description: The system will provide a weekly timetable grid (Mon-Fri between 8AM and 8PM). Students can drag and drop courses from the search results into specific time slots or click on courses to add.

Scenario:

Main: A student drags CSAI 203 Section 1 to Tuesday 10AM. The system will add a box to the timetable with the course code, room number and professor's name.

Alternative: A student clicks on a course block they have added to their timetable and selects Remove.

Exception: A student drops a course block outside the timetable. The system will move the block back to where it was originally.

FR-06: Calculate Cumulative GPA and Semester GPA

Description: The system will calculate the students current Cumulative GPA. It will estimate the students Semester GPA. Users can add expected grades to their current courses to see how it will affect their overall GPA.

Scenario:

Main: A student inputs expected grades (A, B+, A-) for their 5 current courses. The system will instantly calculate their Semester GPA and display it along with showing their new Cumulative GPA.

Alternative: A student updates their predicted grade from B to A. The GPA will instantly update.

Exception: A student inputs an invalid letter grade. An error message will appear and the last calculation will be displayed.

FR-07: Degree Audit

Description: The system will compare the courses a student has taken (and plan to take) against their official major requirements. It will display their progress with a percentage bar and show core, elective and general course lists.

Scenario:

Main: A student accesses the Degree Audit page. The system will display "72 / 130 credits completed" and tick the core courses they have completed.

Alternative: A student adds an elective course to their planned schedule. The progress bar will update to show the elective requirement as Planned.

Exception: A student adds a course that doesn't fit into any of their degree requirements. The course will be listed under Extra Credits and the student will be notified that it won't count towards their graduation.

FR-08: Add to Waiting List

Description: If a course section is full, students can add themselves to the waiting list. The waiting list will rank students based on their priority score. Priority is determined from their seniority (credits completed) and if the course is a required course.

Scenario:

Main: A senior student adds themselves to the waiting list for a required course that is full. The system will calculate their high priority score and add them to the top of the waiting list.

Alternative: A student drops out of the course. The top student on the waiting list will receive an email to claim the spot within 24 hours.

Exception: A student on the waiting list doesn't reply within 24 hours. They will be removed, and the next student will gain the spot.

FR-09: Add New Course

Description: Administrators can add new courses. They can set the number of seats for each section, allocate rooms, set the lecture times and assign professors for the semester.

Scenario:

Main: An admin adds a new section to CSAI 203. They can set the number of seats to 40, assign Dr. Mohamed Sami Rakha and set the time to Mon/Wed 10AM. This new section will immediately be visible to students when searching.

Alternative: An admin updates the number of seats in a section to 40 (from 30). The top 10 students on the waiting list will automatically be offered a seat.

Exception: An admin attempts to assign a professor to two different classes at the same time. The system will stop them saving and display an error message.

FR-10: Export Schedule

Description: Students can download their completed schedule. The system will create a formatted PDF file containing their weekly timetable, courses, room numbers and professors.

Scenario:

Main: A student clicks Export Schedule to PDF. The system will create a clean document.

Alternative: A student selects Landscape. The system will format the PDF.

Exception: A student clicks Export when there are no courses on their calendar. The download button will be disabled and a note will appear telling them to add courses.

FR-11: Submit to Advisor

Description: Students can send their draft schedule to their assigned Academic Advisor (e.g. Dr. Mohamed Sami Rakha). They can also receive feedback.

Scenario:

Main: A student clicks Submit to Advisor. The advisor will receive a notification. They will review the schedule and click Approve.

Alternative: An advisor rejects the schedule because it is too heavy. They will add a comment saying Drop one lab course. The student will receive a notification to make changes.

Exception: A student attempts to submit a schedule with time clashes. The system will stop them and ask them to resolve the clash.

FR-12: Minimum and Maximum Credit Hours

Description: The system has a minimum and maximum credit hour limit (12 & 18). If a student wishes to take extra credits they can automatically submit an Overload Request.

Scenario:

Main: A student with a GPA above 3.5 adds 20 credit hours. The system will detect they have exceeded the 18-credit hour limit. An online form will appear so they can request an overload from the dean.

Alternative: The dean approves the Overload Request. The system will automatically update the students credit hour limit to 20 for that term.

Exception: A student on academic probation attempts to add more than 13 credits. The system will stop them and ask them to visit an advisor.

FR-13: View Past Semesters

Description: The system will store the student's past semesters. They can view their old weekly timetables, courses taken, grades received and past term GPAs.

Scenario:

Main: A student selects Fall 2025 from the drop down menu. The system will load the specific timetable and final grades from that term.

Alternative: An advisor can view a students past history to see if they have repeated courses or changed grades.

Exception: A first semester freshman selects history. The system will display a clear message stating No past terms found. Fall 2026 is your first semester!

FR-14: Toggle Theme

Description: Users can toggle between Light and Dark themes. Students can also customize the background colors of individual course blocks.

Scenario:

Main: A user toggles the theme switch to Dark Mode. The background colours will change to dark grey to help reduce eye strain.

Alternative – Student changes to green colour for Software Engineering. This will store their preference so they can apply it next time they visit.

Exception – User has enabled High Contrast mode. The system will override any course colours.

FR-15: Individual Section Choice

Scenario:

Description – For courses that have lectures and labs/tutorial the student can choose their own individual options for Lectures and Labs ensuring there is no overlap.

Main – Student chooses Lecture Section 1 (Mon 8AM) and Lab Section 3 (Fri 10AM). These are added to their grid.

Alternative – Student only alters their tutorial section.

Exception – Student tries to select an unavailable lab section from another department. The system will prevent this and show they have restrictions.

FR-16: Year Filter

Description – Student can look at courses from different years and levels to get an idea of what they will be doing in the future.

Scenario:

Main – Second year student chooses to view Year 3 courses so they know what courses and prerequisites they will have next year.

Alternative – Student adds a course from the future to the draft schedule to see if it will fit into their long-term schedule.

Exception – Student tries to register for a course that is only available to Seniors. The system will not allow this and show they have restrictions.

FR-17: Professor Filter

Description – Students can search and filter courses by Professor Name to see all the sections they teach during this term.

Scenario:

Main – Student searches by a professor's name. All sections and times they teach are shown.

Alternative – When choosing a professor, it will highlight the classes they offer.

Exception – Professor does not have any available spots. Message will show "All sections for this instructor are currently full".

FR-18: Automatic Conflicts

Description – The system will constantly check to see if there are any conflicts when adding/moving a course section on the draft schedule.

Scenario:

Main – Student tries to add a class that clashes with another class (Mon 10AM vs Mon 10.30AM). The system will not allow this and turn both blocks red. A message will also appear showing there was a conflict.

Alternative– Student has 2 classes scheduled at different buildings with only a 5-minute gap between them. The system will show a non-critical message showing they will have a short travel time.

Exception – Student tries to add 2 different sections of the same course. It will not allow for the second addition and show a message saying Duplicate Course.

FR-19: Local Save (browser Local Storage)

Description – Any changes to the draft schedule will automatically save to local storage on the browser. This ensures the student will not lose any work if the internet drops or the tab is closed.

Scenario:

Main – Student adds some courses. These will automatically save within Half a second.

Alternative – Student accidentally closes their browser. When re-opening the browser, they will have their exact draft schedule.

Exception – Internet drops whilst editing. The system will save locally and show a badge saying Offline - Changes saved locally.

FR-20: Shared URL

Description – The draft schedule can be turned into a unique URL. The student can copy and share this with friends/parents so they can view their schedule.

Scenario:

Main – Student clicks on Share Schedule. It will create a new URL (e.g. planner.zewailcity.edu.eg/s/xK92mP) and copy to their clipboard.

Alternative – Friend opens the URL. It will open a read-only version of the schedule.

Exception – Someone opens a faulty/invalid URL. A message will show saying Invalid Link and ask if they want to open a new schedule.

FR-21: Two Factor Authentication (Admin)

Description – Admins need to use Two Factor Authentication (2FA) to log in to ensure the system is secure.

Scenario:

Main – Admin enters their password, receives a code to their phone and enters this. They can then view the admin panel.

Alternative – Admin needs to verify their identity when carrying out sensitive actions such as deleting a course catalogue.

Exception – Admin enters wrong 2FA code 3 times. This will stop them from logging in and send a security alert.

FR-22: Analytics Dashboard

Description – Admins and advisors can view general stats from the system such as most popular courses, how many seats are filled in each section and how many schedules are awaiting approval.

Scenario:

Main – Admin opens the overview. Charts will be displayed showing which sections are filling up quickly.

Alternative – Advisor opens dashboard to see how many schedules they have awaiting approval from their students.

Exception – Database takes too long to load stats. A basic loading screen will be shown with a retry button.

FR-23: CSV/JSON Upload (Admin)

Description – Admin can upload a CSV/JSON file to add large amounts of courses, rooms, and schedules.

Scenario:

Main– Admin chooses a file with 500 courses. They click import. The file will be validated, and 500 courses will be created in seconds.

Alternative– File has been imported; a report is displayed showing how many records were added.

Exception – CSV file does not contain course code on line 40. It will not import, and no changes will be made to the database. It will highlight they have an issue on line 40.

FR-24: Automated Schedule Generator

Description – Student can choose their target courses and any preferences (e.g. No Morning Classes, No Friday Classes). The system will generate all possible valid schedules without any clashes.

Scenario:

Main – Student chooses 5 courses and selects No Friday Classes. The system will calculate options and find 3 valid schedules. Buttons will appear to preview these schedules.

Alternative – Student clicks Preview Option 2. This will populate the grid with this schedule so it can be saved.

Exception – There are no valid combinations for the chosen courses. The system will stop and show "No valid schedule possible". It will highlight which courses are clashing.

3.4 Non-Functional Requirements

NFR-01: Easy to Use

Description – The UI needs to be simple and easy to use on desktops, tablets and mobiles. Needs to follow basic web accessibility standards.

Test Plan:

Testing – A group of 15 students will try the app on different devices (PC, tablet, mobile). Automated accessibility testing will be carried out using Google Lighthouse.

Success – App gets an average SUS score of 85+. No critical accessibility issues. App displays correctly on mobiles with a minimum width of 360px.

NFR-02 :Client-Side Performance & Responsiveness

Description – The system must process course filtering, drag-and-drop interactions, and conflict-free schedule generation quickly on the browser without freezing or lagging.

Test Plan:

Testing – A sample group of 15 students will run the web application on their personal laptops during the lab session. Browser Developer Tools (Performance Profiler) will be used on each device to measure rendering speed and script execution times.

Success – Schedule generation completes in under 1.5 seconds on all 15 test devices. Drag-and-drop animations run smoothly maintaining 55+ FPS, and local search filtering responds in under 200ms.

NFR-03: Save Draft Schedules

Description – Student's draft schedules need to be saved to ensure they do not lose any work if browser crashes, or internet cuts out.

Test Plan:

Testing – Disconnect internet and force close browser whilst making changes to a schedule. Re-open app to check changes have been saved.

Success – 100% of changes made within 0.5 seconds of crashing will be saved and displayed when re-opening.

NFR-04: Reliable

Description – The system should be reliable and not crash unexpectedly. If an error occurs a friendly error message should show up.

Test Plan:

Testing – Simulate faults within the system (stop database requests, send bad data). Check application handles it correctly.

Success – System will be reliable with an uptime of 99.5%+. 100% of unexpected errors will display a friendly error message with a retry option. No white screen.

NFR-05: Portable

Description – Web Application needs to run exactly the same on all modern browsers (Chrome, Firefox, Safari, Edge). No special browser extensions needed.

Test Plan:

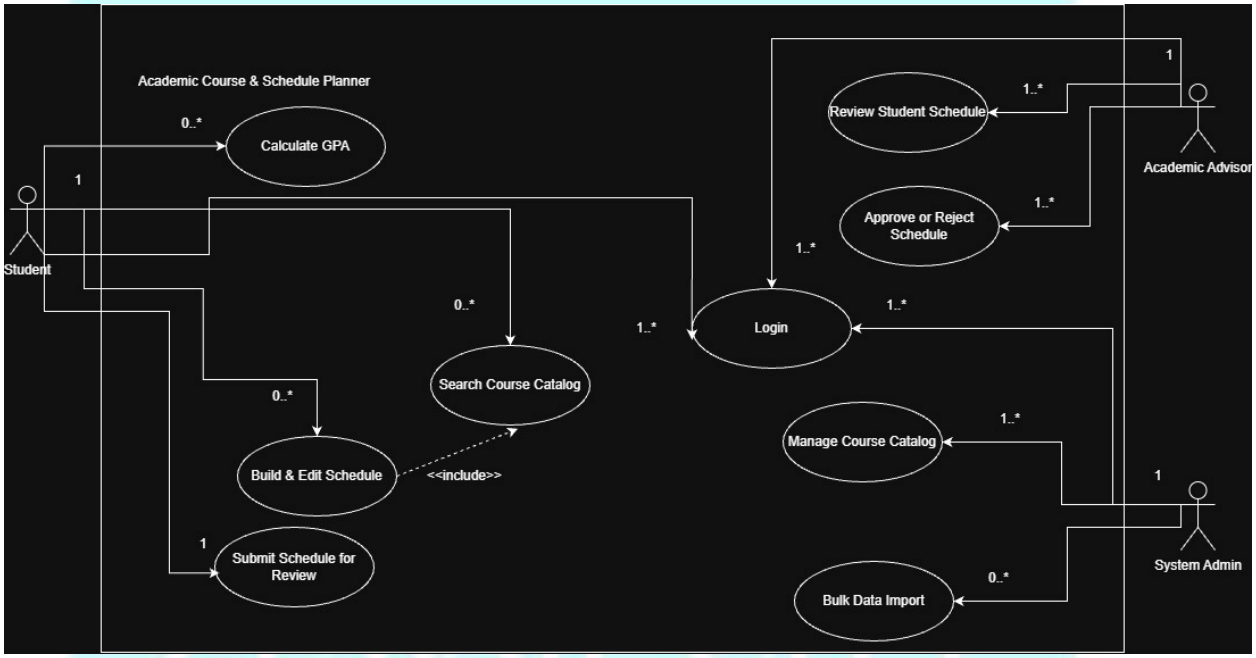
Testing – Automated tests using Playwright will check application works correctly on Chrome, Firefox, Safari and Edge.

Success – 100% of automated tests pass. Application looks the same on all browsers.

3.2 Use Case Model & Descriptions

3.2.1 Use Case Diagram Overview

The Use Case Diagram for the Academic Course & Schedule Planner illustrates the interactions between the system actors (Student, Academic Advisor, and System Admin) and the core system functionalities. It incorporates role-based authentication, multiplicity constraints, and include/extend relationships to enforce secure and structured academic operations.



3.2.2 Use Case Descriptions

UC00: Login (Authenticate User)

- **UC ID: UC00**
- **Use Case Name: Login (Authenticate User)**
- **Actor: Student, Academic Advisor, System Admin**
- **Preconditions: User has a valid institutional account and is on the application login portal.**
- **Postconditions: User identity is verified, and access is granted to the corresponding role-specific dashboard.**
- **Main Flow:**
    1. **User enters institutional username/ID and password.**
    2. **System validates credentials against the secure user database.**
    3. **System authenticates the user and redirects them to their corresponding interface (Student, Advisor, or Admin portal).**

UC01: Search Course Catalog

- **UC ID: UC01**
- **Use Case Name: Search Course Catalog**
- **Actor: Student**
- **Preconditions: Student is logged in and navigating the course lookup tab.**
- **Postconditions: Matching course offerings and real-time seat capacities are displayed.**
- **Main Flow:**
    1. **Student inputs course code, course title, or department filter.**
    2. **System queries the catalog database for matching active course sections.**
    3. **System renders search results along with section schedules, instructors, and prerequisite rules.**

UC02: Build & Edit Schedule

- **UC ID: UC02**
- **Use Case Name: Build & Edit Schedule**
- **Actor: Student**
- **Preconditions: Student is logged in and actively browsing course offerings.**
- **Postconditions: A custom conflict-free schedule draft is updated and saved.**
- **Included Use Case: Search Course Catalog (System automatically invokes catalog search services to fetch available sections during schedule assembly).**
- **Main Flow:**
    1. **Student selects desired course sections to add to the interactive timetable grid.**
    2. **System validates time slots for overlap and checks prerequisite/co-requisite eligibility.**
    3. **System updates the schedule layout and calculates total credit hours.**

UC03: Calculate GPA

- **UC ID: UC03**
- **Use Case Name: Calculate GPA**
- **Actor: Student**
- **Preconditions: Student profile contains historical academic transcripts and current course loads.**
- **Postconditions: Projected semester GPA and cumulative CGPA metrics are displayed.**
- **Main Flow:**
    1. **Student opens the GPA calculator tab and enters target grades for active courses.**

2. System calculates weighted quality points according to institutional grading scales.
3. System outputs estimated term GPA and total CGPA projections.

UC04: Submit Schedule for Review

- UC ID: UC04
- Use Case Name: Submit Schedule for Review
- Actor: Student
- Preconditions: Student has assembled a valid timetable draft with no time conflicts.
- Postconditions: Schedule submission status is updated to "Pending Advisor Review".
- Main Flow:
  1. Student verifies final course selections and clicks "Submit Schedule".
  2. System confirms credit limits and prerequisite compliance.
  3. System forwards the schedule draft to the assigned Academic Advisor's review queue.

UC05: Review Student Schedule

- UC ID: UC05
- Use Case Name: Review Student Schedule
- Actor: Academic Advisor
- Preconditions: Advisor is logged in and has pending student schedule submissions.
- Postconditions: Student schedule draft and academic history are presented for evaluation.
- Main Flow:
  1. Advisor opens the advisor portal and selects a pending student submission.
  2. System displays the proposed timetable, student transcript, and degree audit progress.
  3. Advisor checks credit limits, course sequences, and graduation requirements.

UC06: Approve or Reject Schedule

- UC ID: UC06
- Use Case Name: Approve or Reject Schedule
- Actor: Academic Advisor
- Preconditions: Advisor has reviewed the student's proposed schedule (UC05).
- Postconditions: Schedule status is finalized as "Approved" or "Rejected" with advisory feedback.

- **Main Flow:**
    1. **Advisor selects "Approve" or "Reject" and provides written feedback/comments.**
    2. **System updates submission status in the central database.**
    3. **System dispatches an automated decision notification and comments to the student.**

UC07: Manage Course Catalog

- **UC ID: UC07**
- **Use Case Name: Manage Course Catalog**
- **Actor: System Admin**
- **Preconditions: Admin is logged in with administrative privileges.**
- **Postconditions: Course catalog offerings, seat capacities, and section details are modified.**
- **Main Flow:**
    1. **Admin accesses catalog management to add, update, or remove course sections.**
    2. **System checks input data for scheduling overlaps or room capacity violations.**
    3. **System saves updated course entries into the database.**

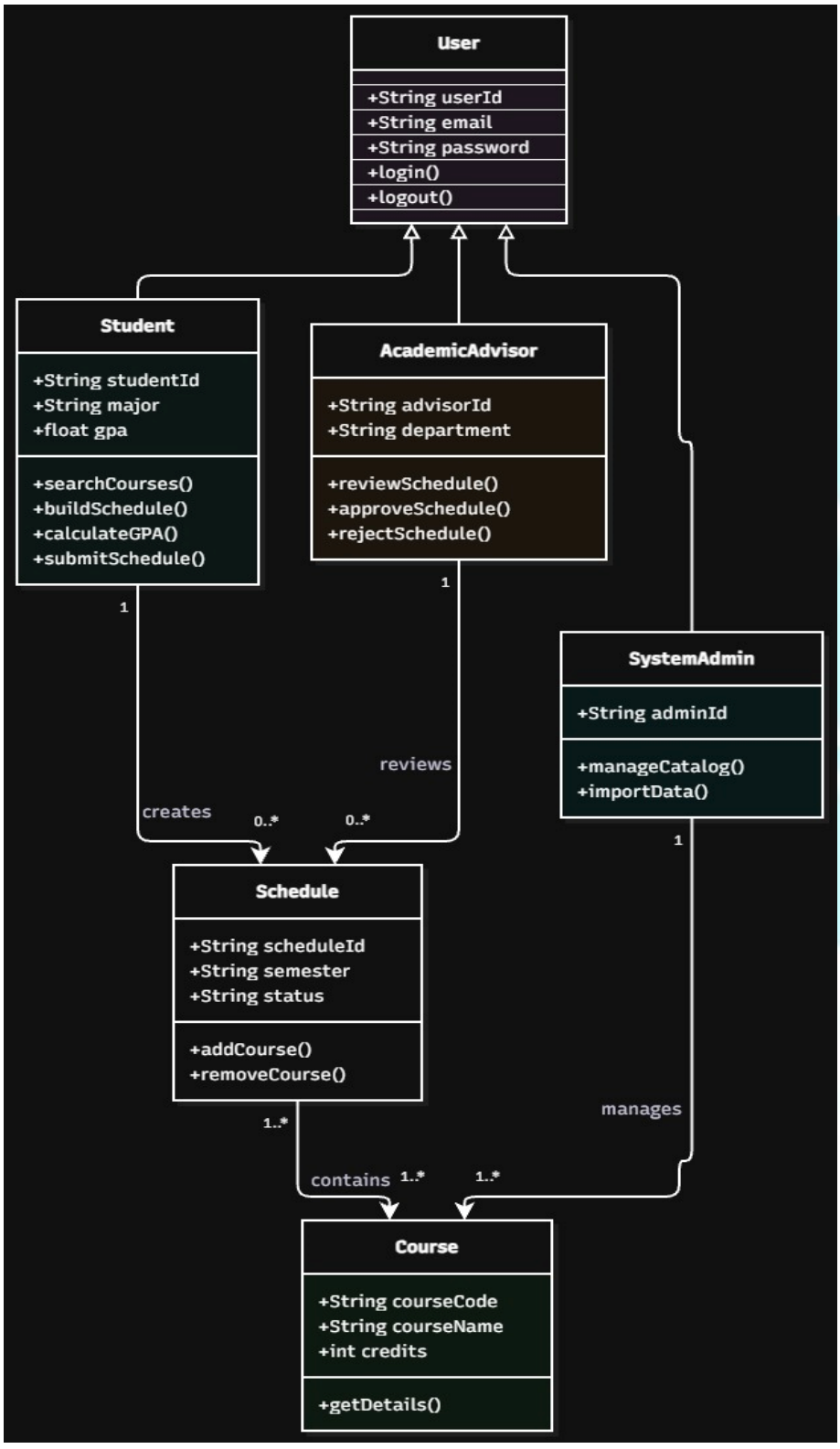
UC08: Bulk Data Import

- **UC ID: UC08**
- **Use Case Name: Bulk Data Import**
- **Actor: System Admin**
- **Preconditions: Admin possesses formatted CSV or JSON data files containing batch records.**
- **Postconditions: System database tables are updated with imported dataset records.**
- **Main Flow:**
    1. **Admin selects and uploads the CSV/JSON file via the admin panel.**
    2. **System parses the file structure and validates record constraints.**
    3. **System populates the database tables and outputs an import summary report.**

3.3 Domain Model & Class Descriptions

3.3.1 Class Diagram Overview

The Domain Class Diagram represents the structural entities of the Academic Course & Schedule Planner, detailing their attributes, operations, inheritance hierarchies, and relational multiplicities.



3.3.2 Class Descriptions

1. **User (Base Class):**
    - **Description: An abstract base entity representing any authenticated individual accessing the system.**
    - **Attributes: userId (String), email (String), password (String).**
    - **Methods: login(), logout().**
2. **Student (Subclass of User):**
    - **Description: Represents undergraduate students using the platform to construct schedules, search courses, and monitor academic progress.**
    - **Attributes: studentId (String), major (String), gpa (Float).**
    - **Methods: searchCourses(), buildSchedule(), calculateGPA(), submitSchedule().**
3. **AcademicAdvisor (Subclass of User):**
    - **Description: Represents faculty advisors tasked with reviewing and approving student schedule proposals.**
    - **Attributes: advisorId (String), department (String).**
    - **Methods: reviewSchedule(), approveSchedule(), rejectSchedule().**
4. **SystemAdmin (Subclass of User):**
    - **Description: Represents system managers responsible for catalog maintenance and data batch imports.**
    - **Attributes: adminId (String).**
    - **Methods: manageCatalog(), importData().**
5. **Course:**
    - **Description: Represents individual academic course offerings listed in the university catalog.**
    - **Attributes: courseCode (String), courseName (String), credits (Integer).**
    - **Methods: getDetails().**
6. **Schedule:**
    - **Description: Represents a student's drafted or submitted semester timetable proposal.**
    - **Attributes: scheduleId (String), semester (String), status (String).**
    - **Methods: addCourse(), removeCourse().**

3.5 External Interface Requirements

3.5.1 User Interfaces

- **Responsive Layout: The application provides a clean, responsive user interface designed to scale seamlessly across desktops, tablets, and mobile devices, supporting a minimum horizontal screen resolution of 360px.**
- **Role-Based Portals: Upon successful authentication (FR-01), users are automatically redirected to dedicated, role-specific dashboards tailored for Students, Academic Advisors, and System Administrators.**
- **Interactive Timetable Grid: The student schedule workspace features a weekly drag-and-drop grid (Monday–Friday, 8:00 AM – 8:00 PM) with real-time visual conflict indicators, theme toggling (Light/Dark mode with custom course block coloring), and inline error messaging.**
- **Accessibility Standards: Built in compliance with fundamental web accessibility standards, ensuring clear navigation, helpful contextual tooltips, and readable high-contrast elements.**

3.5.2 Hardware Interface

- **Client-Side Hardware: Standard desktop computers, laptops, tablets, or mobile smartphones equipped with standard input peripherals (mouse, keyboard, or touchscreen interfaces).**
- **Server-Side Infrastructure: Centralized web application server architecture hosting the backend logic and maintaining continuous connectivity with the database storage layer.**

3.5.3 Software Interface

- **Client-Side Environment: Execution relies entirely on standard, modern web browsers without requiring any special browser extensions or plugins, including Google Chrome (90+), Mozilla Firefox (88+), Microsoft Edge, and Safari.**
- **Server-Side Framework: Developed using Python (Flask framework) or C# (.NET Core) to handle routing, business logic, and request processing.**
- **Database Management System (DBMS): Relational database layer (such as PostgreSQL or SQLite) utilized for persistent storage of user accounts, course catalogs, student transcripts, and academic schedule records.**
- **Browser LocalStorage: Integrated client-side storage mechanism to automatically back up and persist schedule drafts locally within 0.5 seconds, ensuring offline recovery and data protection during network disruptions.**

3.5.4 Communications Interface

- **Secure Protocols: All data communication between the client browser and the server is secured via Hypertext Transfer Protocol Secure (HTTPS / TLS 1.3 encryption), protecting user credentials, administrative Two-Factor Authentication (2FA) tokens, and academic records.**
- **API & Data Exchange: RESTful API architecture designed for internal modular data flows, file batch imports (CSV/JSON datasets for course catalogs), and structured URL sharing for draft schedules (planner.zewailcity.edu.eg/s/...).**
- **System Integration Boundaries: Designed with flexible API boundaries to support future enterprise integration with Zewail City's official university Student Information Systems (SIS) and centralized single sign-on (SSO) authentication portals.**

4. Appendices

4.1 Appendix A: Data Dictionary

The Data Dictionary provides formal definitions and data types for the core data elements utilized within the Academic Course & Schedule Planner database architecture.

Data Element Name	Data Type	Description & Constraints
userId	String (UUID)	Unique system identifier assigned to every registered user (Primary Key).
email	String	Institutional email address used for authentication (Unique, non-null).
password	String	Encrypted user password hash (salted and securely stored).
role	Enumeration	Defines user permissions (Student, AcademicAdvisor, SystemAdmin).
studentId	String	Unique university identification number for students.

Data Element Name	Data Type	Description & Constraints
major	String	Primary academic program or major field of study (e.g., Software Development).
gpa / cgpa	Float	Semester grade point average or cumulative GPA (Range: 0.00 to 4.00).
advisorId	String	Unique identification number assigned to faculty advisors.
courseCode	String	Standardized alphanumeric course identifier (e.g., CSAI 203).
courseName	String	Full descriptive title of the academic course.
credits	Integer	Credit hour weight assigned to a specific course (Typical range: 1 to 4).
scheduleId	String (UUID)	Unique identifier for a student's semester timetable draft or submission.
semester	String	Academic term designation (e.g., Fall 2026, Spring 2027).
status	Enumeration	Schedule approval status (Pending, Approved, Rejected).

4.2 Appendix B: Glossary

This glossary defines key technical, academic, and architectural terms used throughout this Software Requirements Specification document.

- **API (Application Programming Interface): A set of protocols and routines enabling different software applications to communicate and exchange data securely.**
- **Co-requisite: An academic course that must be taken concurrently with another specific course during the same semester.**

- **Degree Progress Audit: An automated evaluation system comparing a student's completed and planned coursework against institutional graduation requirements.**
- **MVC (Model-View-Controller): A software architectural pattern separating an application into three main components: data models, user interface views, and control logic.**
- **Prerequisite: A mandatory foundational course that a student must successfully pass before registering for an advanced course.**
- **RDBMS (Relational Database Management System): A database management system that stores data in structured, table-based formats with defined relations (e.g., PostgreSQL, SQLite).**
- **SRS (Software Requirements Specification): A comprehensive document outlining the functional behaviors, system constraints, and external interfaces of a software product.**
- **SUS (System Usability Scale): A standardized industry questionnaire used to measure subjective perceptions of system usability and user experience.**
- **Two-Factor Authentication (2FA): An identity verification method requiring users to provide two distinct authentication factors to gain system access (mandatory for Administrators).**

ZEWAIL CITY
ESTABLISHED 2000
INAUGURATED 2011